

Internship & Recruitment Management Platform

MERN + Python AI Microservice — Full Project Architecture, Folder Structure & Phase-by-Phase Build Plan

Quality Manager	Bijon
Project Manager	Thuva
Risk Manager	Maarisinsh
Startup / Schedule Manager	Divagar
Stack	MongoDB, Express, React, Node.js + Python (FastAPI)
Estimated Build Time	3 weeks (initial version)

1. Why MERN + Python (Not MERN Alone)

The original scope (login, dashboards, quiz, applications, interview scheduling) fits MERN perfectly — it's standard CRUD + auth. The **new scope** changes that:

- **CV filtering across 20,000 resumes** — needs NLP (resume parsing, skill extraction, ranking). Node has no mature ecosystem for this.
- **Auto resource allocation** — needs a scoring/optimization algorithm (workload + skill match). Easier and faster to build in Python.

Decision: Keep MERN as the main application (auth, dashboards, workflow, attendance, scheduling). Build a **separate Python service** just for AI tasks (CV parsing/ranking, allocation engine). Node calls Python over a private internal REST API. This is the same pattern companies like LinkedIn and Workday use internally — one stack per job, not one stack forcing every job.

Layer	Tech	Why
Frontend	React.js + Redux Toolkit	Two dashboards (Interviewer/Candidate) + new HR dashboards
Backend (Main API)	Node.js + Express	Auth, CRUD, workflow, attendance, scheduling
Database	MongoDB + Mongoose	Flexible schema — good fit for varying CV/internship fields
AI/CV Service	Python + FastAPI	CV parsing, ranking, resource-allocation scoring
AI Libraries	spaCy, pdfplumber, scikit-learn	Resume text extraction + similarity scoring (cheap, no paid API needed)
Real-time	Socket.io	Live attendance status, interview reminders, application status push
Auth	JWT + bcrypt	Two role types (Interviewer/Candidate) + later Employee/Manager roles
File Storage	Cloudinary / AWS S3	Resume PDFs, profile photos
Video Call	Zoom SDK / Jitsi embed	Face-to-face round must open inside the system, per client
Deployment	Render/Railway (Node) + Render (Python) + Vercel (React)	Free-tier friendly, matches H2F's existing hosting approach

Note: Python is not optional add-on — it is the only realistic way to filter 20,000 CVs without buying an expensive AI API. spaCy + scikit-learn run locally, free, and fast enough for batch filtering.

2. System Modules (Full Scope)

Module	Belongs To	Key Features
Auth & Roles	Node	Interviewer, Candidate, Employee, HR Manager — signup/login/JWT
Quiz Builder	Node	Create quiz sets, titles, saved as list, reusable across internships
Post Internship	Node	Company details, internship details, timeline, assessment config, quiz dropdown
Candidate Portal	Node	Browse internships (cards), apply, track application status
Application Pipeline	Node	Applied → Reviewed → Quiz → Face-to-Face → Hired/Rejected
Face-to-Face Scheduling	Node	In-system meeting link, visible on both dashboards
CV Bulk Filtering	Python	Upload/parse 20,000 CVs, extract skills, rank by job match
Employee Workflow Tracking	Node	Employee → Project → Task, status (Not Started/In Progress/Done)
Attendance (Clock-in/out)	Node	Web-based, timestamped, daily logs, reports
Auto Resource Allocation	Python	New project → auto-suggest employees based on current workload + skill
Admin/HR Dashboard	Node	Overview of all employees, projects, attendance, allocation overrides

3. Database Schema (MongoDB Collections)

Collection	Key Fields
users	name, email, passwordHash, role (interviewer/candidate/employee/admin), createdAt
quizzes	title, createdBy, questions[], createdAt

internships	companyDetails, internshipDetails, timeline, quizId, assessmentConfig, postedBy
applications	candidateId, internshipId, status, resumeUrl, quizScore, interviewSlot
cvbank	candidateName, resumeUrl, extractedSkills[], rankScore, tags (Python writes here)
projects	name, client, requiredSkills[], startDate, status
tasks	projectId, assignedTo, title, status, hoursLogged
attendance	employeeId, clockIn, clockOut, date, totalHours
allocations	projectId, suggestedEmployees[], finalAssigned[], overriddenBy

4. Folder Structure

4.1 Root Structure (Monorepo)

```
internship-platform/
├─ client/                # React frontend
├─ server/                # Node.js + Express backend
├─ ai-service/            # Python FastAPI (CV filtering + allocation)
├─ docs/                  # SRS, ER diagrams, API docs
├─ .gitignore
└─ README.md
```

4.2 client/ (React)

```
client/
├─ public/
├─ src/
│  └─ api/                # axios instances per service
│     ├── authApi.js
│     ├── internshipApi.js
│     └─ aiApi.js         # calls ai-service via Node proxy
│  └─ assets/
│  └─ components/
│     ├── common/         # Button, Modal, Loader, Navbar
│     ├── interviewer/    # QuizBuilder, PostInternshipForm, ApplicantList
│     ├── candidate/      # InternshipCard, ApplicationStatus
│     └─ hr/              # AttendanceWidget, AllocationBoard, TaskBoard
│  └─ pages/
│     ├── auth/           # Login, Signup
│     ├── interviewer/    # Dashboard, Questions, PostInternship, Applications, Interviews
│     ├── candidate/      # Dashboard, Internships, AppliedInternships
│     └─ hr/              # EmployeeDashboard, ProjectTracker, AttendancePage
│  └─ redux/
│     ├── store.js
│     └─ slices/          # authSlice, internshipSlice, attendanceSlice
│  └─ routes/
│     ├── PrivateRoute.jsx
│     └─ AppRoutes.jsx
│  └─ utils/
│     ├── App.jsx
│     └─ main.jsx
├─ package.json
└─ vite.config.js
```

4.3 server/ (Node + Express)

```
server/
├─ src/
│  └─ config/
│     ├── db.js           # MongoDB connection
│     └─ env.js
│  └─ models/
│     ├── User.js
│     ├── Quiz.js
│     ├── Internship.js
│     ├── Application.js
│     ├── Project.js
│     ├── Task.js
│     ├── Attendance.js
│     └─ Allocation.js
│  └─ controllers/
│     ├── authController.js
│     ├── quizController.js
│     ├── internshipController.js
│     ├── applicationController.js
│     ├── interviewerController.js
│     └─ attendanceController.js
```

```

├── taskController.js
├── allocationController.js # proxies to Python
├── routes/
│   ├── authRoutes.js
│   ├── quizRoutes.js
│   ├── internshipRoutes.js
│   ├── applicationRoutes.js
│   ├── attendanceRoutes.js
│   ├── taskRoutes.js
│   └── allocationRoutes.js
├── middleware/
│   ├── authMiddleware.js # JWT verify
│   ├── roleMiddleware.js # role-based access
│   └── errorHandler.js
├── services/
│   └── aiServiceClient.js # axios calls to Python ai-service
├── sockets/
│   └── index.js # Socket.io events
├── app.js
├── server.js
└── package.json

```

4.4 ai-service/ (Python + FastAPI)

```

ai-service/
├── app/
│   ├── main.py # FastAPI entrypoint
│   ├── routers/
│   │   ├── cv_filter.py # /filter-cvs endpoint
│   │   └── allocation.py # /suggest-allocation endpoint
│   └── services/
│       ├── resume_parser.py # pdfplumber/docx text extraction
│       ├── skill_extractor.py # spaCy NER + keyword matching
│       ├── ranking_engine.py # TF-IDF / cosine similarity scoring
│       └── allocation_engine.py # workload + skill-match scoring
├── models/
│   └── schemas.py # Pydantic request/response models
├── db/
│   └── mongo_client.py # shared MongoDB connection (pymongo)
├── requirements.txt
└── Dockerfile

```

5. Phase-by-Phase Build Plan

Total: 3 weeks. Each phase has clear owner and deliverable so nothing overlaps.

Phase 0 — Setup & Finalize Scope Day 1-2

- Lock requirements doc (this PDF) with client — confirm CV filter criteria, attendance mode (GPS or not), allocation logic (workload-only vs skill-match)
- Repo setup: `client/`, `server/`, `ai-service/` initialized, pushed to GitHub
- MongoDB Atlas cluster created, environment variables shared securely
- Figma wireframes for Interviewer, Candidate, and HR dashboards

Owner: Divagar (setup) + Bijon (wireframe QA)

Phase 1 — Auth & Core Models Day 3-4

- User model (4 roles), signup/login, JWT middleware, role-based route guards
- React auth pages + PrivateRoute setup + Redux authSlice
- Base layouts for 3 dashboards (Interviewer / Candidate / HR)

Owner: Thuva

Phase 2 — Quiz Builder + Post Internship Day 5-7

- Quiz model + CRUD, quiz list saved per interviewer
- Post-internship form (company/internship details, timeline, assessment config, quiz dropdown) — client to confirm exact field list/dropdowns
- Internship cards on candidate side, public listing

Owner: Thuva + Divagar

Phase 3 — Application Pipeline Day 8-9

- Apply flow, resume upload (Cloudinary), application status states
- Interviewer review screen (accept/reject)
- Quiz attempt flow + auto pass/fail logic

Owner: Maarisinsh (flow logic + edge cases)

Phase 4 — Face-to-Face Scheduling Day 10-11

- In-system scheduling, embedded meeting link (Zoom SDK/Jitsi)
- Visible on both dashboards, Socket.io reminder notification

Owner: Thuva

Phase 5 — Python AI Service: CV Filtering Day 10-13 (parallel)

- FastAPI service scaffold, MongoDB connection shared with main DB
- Resume parser (PDF/DOCX → text), skill extractor (spaCy)
- Ranking engine (TF-IDF/cosine similarity against job requirements)
- Bulk upload endpoint — tested against sample batch before full 20,000
- Node calls Python via internal API, results stored in `cvbank` collection

Owner: Bijon (quality/accuracy validation) + Divagar (infra)

Phase 6 — Workflow, Attendance, Allocation Day 12-15

- Project/Task models + Kanban-style status board
- Clock-in/clock-out UI + backend timestamp logging + daily report
- Allocation engine (Python): workload score + skill match → suggests employees for new project
- Manager override UI on allocation suggestions

Owner: Maarisinsh (risk: GPS scope, override logic) + Bijon

Phase 7 — Integration, Testing, QA Day 16-18

- End-to-end flow test: candidate apply → quiz → interview → hired → becomes employee → clocks in → assigned to project
- Load test CV filter with full 20,000 dataset (batch processing, not real-time)
- Bug fixes, role-permission audit

Owner: Bijon (full QA sign-off)

Phase 8 — Deployment & Handover Day 19-21

- Deploy: React → Vercel, Node → Render/Railway, Python → Render, MongoDB Atlas (prod)
- Domain + SSL setup, env secrets locked down
- Client demo + walkthrough, handover docs, maintenance terms confirmed

Owner: Divagar

6. Step-by-Step Build Order (Technical)

1. Init monorepo → `git init`, create 3 folders, add root `.gitignore`
2. `server/`: `npm init -y` → install express, mongoose, jsonwebtoken, bcryptjs, dotenv, cors, socket.io
3. Connect MongoDB Atlas, build User model, test signup/login with Postman
4. `client/`: `npm create vite@latest client -- --template react` → install redux toolkit, axios, react-router-dom, tailwindcss
5. Build auth pages, connect to backend, store JWT in httpOnly cookie or localStorage
6. Build Quiz + Internship CRUD (Node) → connect to React forms
7. Build Application pipeline state machine (Applied→Reviewed→Quiz→Interview→Hired/Rejected)
8. `ai-service/`: `python -m venv venv` → install fastapi, uvicorn, pdfplumber, spacy, scikit-learn, pymongo
9. Build `/filter-cvs` endpoint → test with 50 sample resumes first, then scale
10. Build `/suggest-allocation` endpoint → test with mock employee workload data
11. Connect Node → Python via internal axios calls (`aiServiceClient.js`)
12. Build Attendance + Task tracking modules (pure Node/React, no AI needed)
13. Integrate Socket.io for real-time status updates
14. Full integration test → deploy each service → final client demo

7. Open Questions for Client (Before Coding Starts)

- CV filter criteria: skills only, or also experience years/education/location?
- CVs: one-time bulk import or continuous candidate uploads?
- Attendance: GPS-based clock-in required, or simple timestamp is enough?
- Allocation: pick lowest workload only, or workload + skill match?
- Can a manager manually override an auto-suggested allocation?
- Is the new HR/workflow system part of the same platform as the Interview Management System, or a separate product?